



UNIVERSIDAD ESTATAL DE MILAGRO

**FACULTAD DE CIENCIAS E INGENIERÍA
CARRERA DE INGENIERÍA DE SOFTWARE**

TEMA:

BÚSQUEDA NO INFORMADA, INFORMADA Y AG

AUTORES:

OSWALDO DANILO ANGULO TAMAYO

FERNANDO JAVIER ZAVALA CATAÑA

FRANCISCO STEVEN ARIAS PEREZ

DARLY DOUGLAS FARIAS MENDOZA

ASIGNATURA:

INTELIGENCIA ARTIFICIAL

DOCENTE:

ING.JHONNY DARWIN ORTIZ MATA

FECHA DE ENTREGA:

31/05/2026

PERIODO:

Abril 2026 a Julio 2026

EJERCICIO 1

Búsqueda en amplitud de A hasta Y

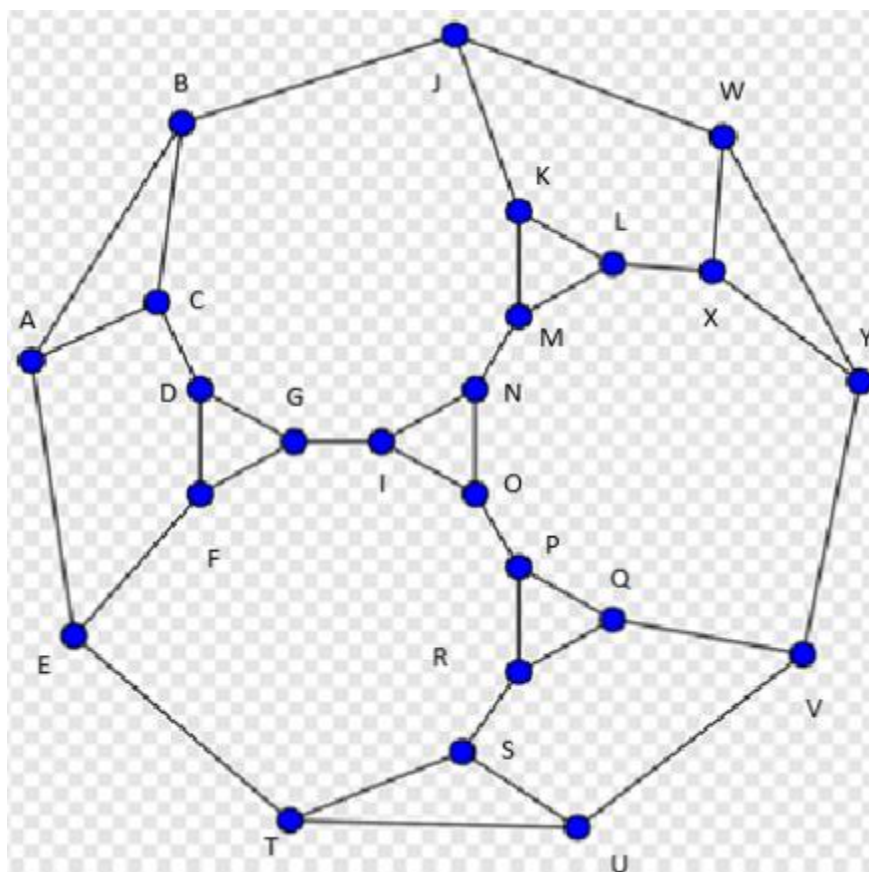


figura 1 ejercicio 1

lista de conexiones

$A = \{B, c, e\}$	$B = \{A, c, \mathcal{I}\}$	$C = \{A, B, d\}$
$d = \{c, f, G\}$	$e = \{A, e, T\}$	$f = \{e, D, B\}$
$G = \{D, f, \mathcal{I}\}$	$\mathcal{I} = \{G, N, O\}$	$\mathcal{I} = \{B, K, w\}$
$K = \{\mathcal{I}, L, H\}$	$L = \{K, H, x\}$	$H = \{K, L, N\}$
$N = \{\mathcal{I}, H, O\}$	$O = \{\mathcal{I}, N, P\}$	$P = \{O, R, Q\}$
$Q = \{P, R, v\}$	$R = \{S, P, Q\}$	$S = \{T, e, u\}$
$T = \{e, s, u\}$	$u = \{T, S, v\}$	$v = \{u, Q, y\}$
$w = \{\mathcal{I}, x, y\}$	$x = \{L, w, y\}$	$y = \{w, x, v\}$

figura 2 lista de conexiones

Inicio A	Fin Y
Estado	Expandido
$\{A\}$	$\{A\}$
$\{BA, CA, EA\}$	$\{BA\}$
$\{CA, EA, JB\}$	$\{CA\}$
$\{EA, JB, DC\}$	$\{EA\}$
$\{JB, DC, FE, TE\}$	$\{JB\}$
$\{DC, FE, TE, KJ, WJ\}$	$\{DC\}$
$\{FE, TE, KJ, WJ, GD\}$	$\{FE\}$
$\{TE, KJ, WJ, GD\}$	$\{TE\}$
$\{KJ, WJ, GD, ST, MT\}$	$\{KJ\}$
$\{WJ, GD, ST, MT, LK, MK\}$	$\{WJ\}$
$\{GD, ST, MT, LK, MK, XW, YW\}$	$\{GD\}$
$\{ST, MT, LK, MK, XW, YW, IG\}$	$\{ST\}$
$\{MT, LK, MK, XW, YW, IG, PS\}$	$\{MT\}$
$\{LK, MK, XW, YW, IG, PS, VA\}$	$\{LK\}$

figura 3 estados-Expandidos

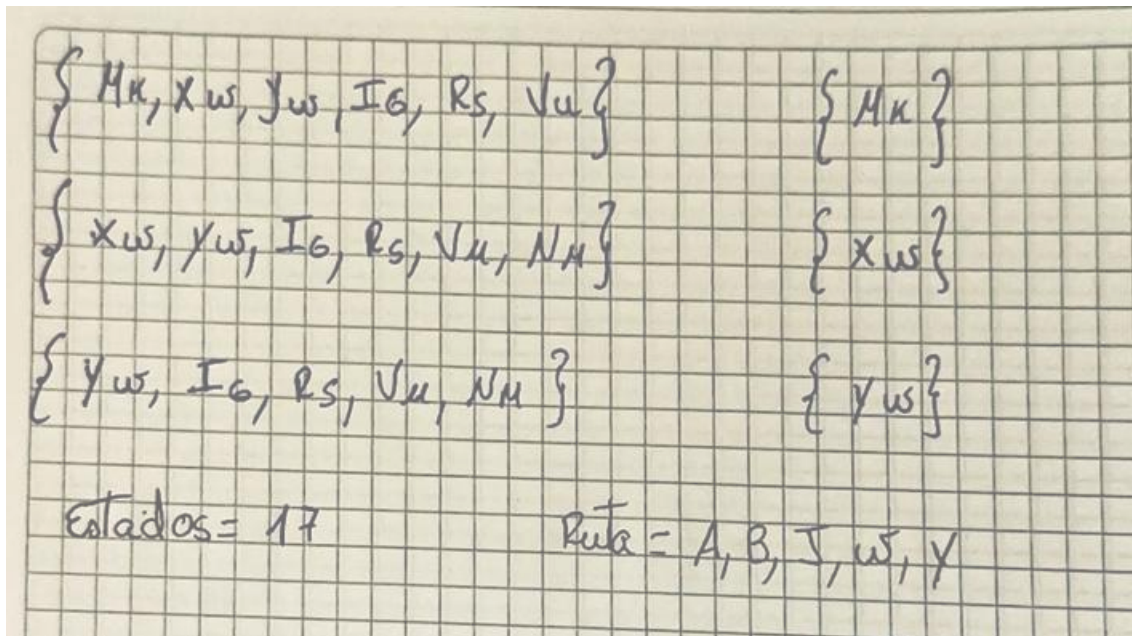


figura 4 resultado

```
def busqueda_amplitud(grafo, inicio, fin):
    cola = [(inicio, [inicio])]
    # Set para llevar el control de los nodos que ya entraron a la cola
    visitados = {inicio}

    estados_expandidos = 0
    orden_expansion = []

    while cola:
        # Extraemos el primer elemento de la cola
        nodo_actual, ruta = cola.pop(0)

        # Al sacarlo de la cola, lo consideramos un estado "expandido"
        estados_expandidos += 1
        orden_expansion.append(nodo_actual)

        # Si el nodo actual es nuestro objetivo detenemos la busqueda
        if nodo_actual == fin:
            return ruta, estados_expandidos, orden_expansion

        # Revisamos las conexiones
        for vecino in grafo[nodo_actual]:
            if vecino not in visitados:
                visitados.add(vecino)
                cola.append((vecino, ruta + [vecino]))

    return None, estados_expandidos, orden_expansion
```

figura 5 Búsqueda amplitud

```
# 1.lista de conexiones
grafo_conexiones = {
    'A': ['B', 'C', 'E'],
    'B': ['A', 'C', 'J'],
    'C': ['A', 'B', 'D'],
    'D': ['C', 'F', 'G'],
    'E': ['A', 'F', 'T'],
    'F': ['E', 'D', 'G'],
    'G': ['D', 'F', 'I'],
    'I': ['G', 'N', 'O'],
    'J': ['B', 'K', 'W'],
    'K': ['J', 'L', 'M'],
    'L': ['K', 'M', 'X'],
    'M': ['K', 'L', 'N'],
    'N': ['I', 'M', 'O'],
    'O': ['I', 'N', 'P'],
    'P': ['O', 'R', 'Q'],
    'Q': ['P', 'R', 'V'],
    'R': ['S', 'P', 'Q'],
    'S': ['T', 'R', 'U'],
    'T': ['E', 'S', 'U'],
    'U': ['T', 'S', 'V'],
    'V': ['U', 'Q', 'Y'],
    'W': ['J', 'X', 'Y'],
    'X': ['L', 'W', 'Y'],
    'Y': ['W', 'X', 'V']
}

ruta_final, total_estados, nodos_expandidos = busqueda_amplitud(grafo_conexiones, 'A', 'Y')

# 3. resultados
print(f"Estados expandidos: {total_estados}")
print(f"Ruta óptima encontrada: {' -> '.join(ruta_final)}")

Estados expandidos: 17
Ruta óptima encontrada: A -> B -> J -> W -> Y
```

figura 6 resultado colab

EJERCICIO 2

En el siguiente árbol realice la búsqueda de la solución del estado n_1 hasta n_{10} aplicando: • A*

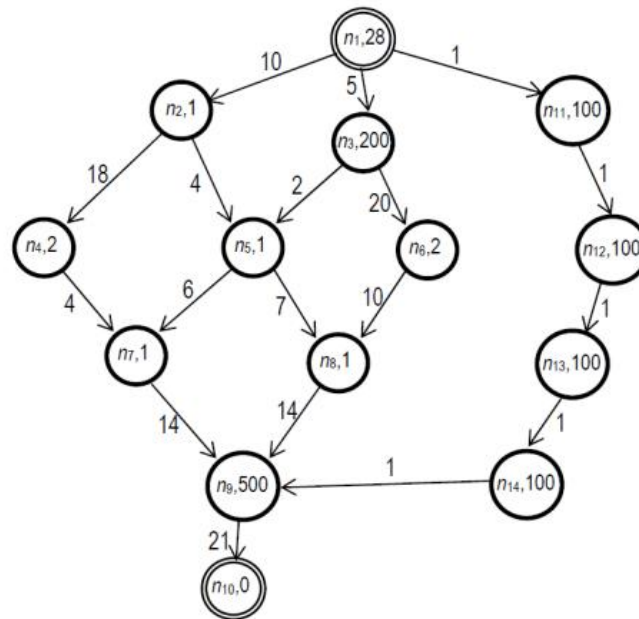


figura 7 ejercicio 2

N_1	N_1
$\{N_2N_1, N_3N_1, N_{11}N_1\}$	$\{N_2N_1\}$
$\{N_4N_2, N_5N_2\}$	$\{N_5N_2\}$
$\{N_7N_5, N_8N_5\}$	$\{N_7N_5\}$
$\{N_9N_7\}$	$\{N_8N_7\}$
$\{N_9N_8\}$	$\{N_4N_2\}$
$\{N_7N_4\}$	$\{N_{11}N_1\}$
$\{N_{12}N_{11}\}$	$\{N_{12}N_{11}\}$
$\{N_{13}N_{12}\}$	$\{N_{13}N_{12}\}$
$\{N_{14}N_{13}\}$	$\{N_{14}N_{13}\}$
$\{N_9N_{14}\}$	$\{N_9N_{14}\}$
$\{N_{10}N_9\}$	$\{N_{10}N_9\}$

Distancia = 26 Ruta = $\{N_1, N_{11}, N_{12}, N_{13}, N_{14}, N_9, N_{10}\}$

figura 8 resultado ejercicio 2

```
import heapq

def busqueda_a_estrella(grafo, heuristica, inicio, fin):

    cola = [(heuristica[inicio], 0, inicio, [inicio])]

    costo_g_minimo = {inicio: 0}

    while cola:
        # Extraemos el nodo con el menor f_score de la cola
        f_actual, g_actual, nodo_actual, ruta = heapq.heappop(cola)

        if nodo_actual == fin:
            return ruta, g_actual

        # Exploramos las conexiones (vecinos) del nodo actual
        for vecino, costo_arista in grafo[nodo_actual].items():

            nuevo_g = g_actual + costo_arista

            if vecino not in costo_g_minimo or nuevo_g < costo_g_minimo[vecino]:
                costo_g_minimo[vecino] = nuevo_g

            f_vecino = nuevo_g + heuristica[vecino]

            # Agregamos el vecino a la cola de prioridad
            heapq.heappush(cola, (f_vecino, nuevo_g, vecino, ruta + [vecino]))

    return None, 0
```

```
# 1. Definimos el grafo con los costos de las aristas g(n)
grafo = {
    'n1': {'n2': 10, 'n3': 5, 'n11': 1},
    'n2': {'n4': 18, 'n5': 4},
    'n3': {'n5': 2, 'n6': 20},
    'n4': {'n7': 4},
    'n5': {'n7': 6, 'n8': 7},
    'n6': {'n8': 10},
    'n7': {'n9': 14},
    'n8': {'n9': 14},
    'n9': {'n10': 21},
    'n10': {},
    'n11': {'n12': 1},
    'n12': {'n13': 1},
    'n13': {'n14': 1},
    'n14': {'n9': 1}
}

# 2. Definimos la heurística h(n) de cada nodo (los valores dentro de los círculos)
heuristica = {
    'n1': 28, 'n2': 1, 'n3': 200, 'n4': 2, 'n5': 1,
    'n6': 2, 'n7': 1, 'n8': 1, 'n9': 500, 'n10': 0,
    'n11': 100, 'n12': 100, 'n13': 100, 'n14': 100
}

# 3. Ejecutamos el algoritmo desde 'n1' hasta 'n10'
ruta_optima, costo_total = busqueda_a_estrella(grafo, heuristica, 'n1', 'n10')

# 4. resultados
print(f"Ruta óptima (R): [{', '.join(ruta_optima)}]")
print(f"Costo total (V): {costo_total}")

Ruta óptima (R): [n1, n11, n12, n13, n14, n9, n10]
Costo total (V): 26
```

figura 9 resultado colab

EJERCICIO 3

En el siguiente árbol realice la búsqueda de la solución desde H hasta E aplicando:

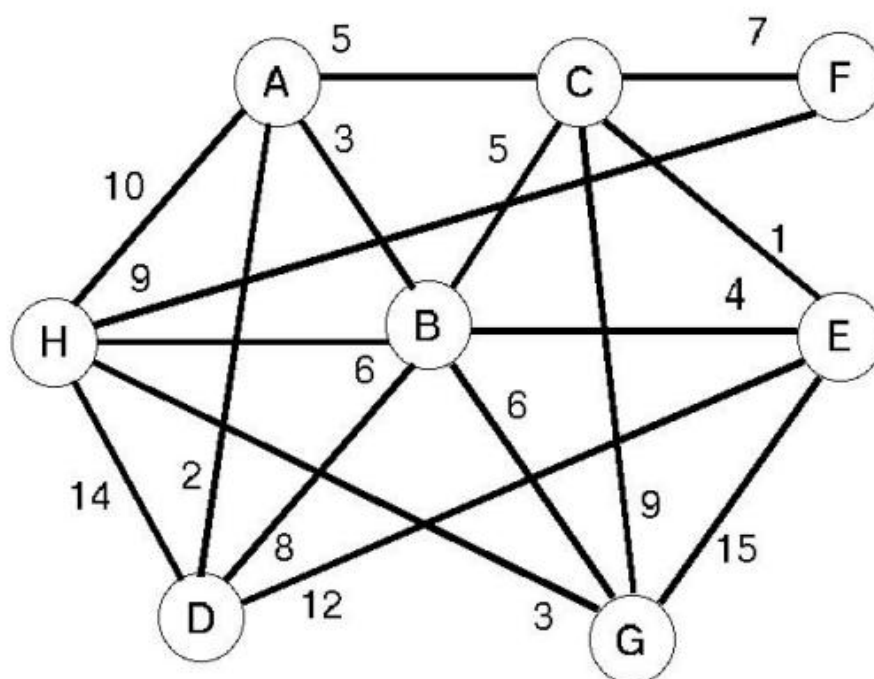


figura 10 ejercicio 3



figura 11 resultado

```
# Definición de la estructura para los nodos (la misma de siempre)
class Nodo:
    def __init__(self, datos, hijos=None):
        self.datos = datos
        self.hijos = None
        self.padre = None
        self.coste = None
        self.set_hijos(hijos)

    def set_hijos(self, hijos):
        self.hijos = hijos
        if self.hijos is not None:
            for h in self.hijos:
                h.padre = self

    def get_hijos(self):
        return self.hijos

    def get_padre(self):
        return self.padre

    def set_padre(self, padre):
        self.padre = padre

    def set_datos(self, datos):
        self.datos = datos

    def get_datos(self):
        return self.datos

    def set_coste(self, coste):
        self.coste = coste

    def get_coste(self):
        return self.coste

    def igual(self, nodo):
        return self.get_datos() == nodo.get_datos()

    def en_lista(self, lista_nodos):
        for n in lista_nodos:
            if self.igual(n):
                return True
        return False

    def __str__(self):
        return str(self.get_datos())
```

figura 12 ejecución de ejercicio en colab

```
# Función para la búsqueda de coste uniforme
def buscar_solucion_UCS(conexiones, estado_inicial, solucion):
    solucionado = False
    nodos_visitados = []
    nodos_frontera = []

    nodo_inicial = Nodo(estado_inicial)
    nodo_inicial.set_coste(0)
    nodos_frontera.append(nodo_inicial)

    while (not solucionado) and len(nodos_frontera) != 0:
        # Ordenamos por coste para sacar siempre el más barato
        nodos_frontera.sort(key=lambda x: x.get_coste())

        nodo = nodos_frontera.pop(0)
        nodos_visitados.append(nodo)

        if nodo.get_datos() == solucion:
            solucionado = True
            return nodo
        else:
            # Expandimos los hijos según las conexiones del grafo
            dato_nodo = nodo.get_datos()
            lista_hijos = []

            for un_hijo in conexiones[dato_nodo]:
                coste = conexiones[dato_nodo][un_hijo]
                hijo = Nodo(un_hijo)
                hijo.set_coste(nodo.get_coste() + coste)
                lista_hijos.append(hijo)

                if not hijo.en_lista(nodos_visitados):
                    # Si ya está en la lista, vemos si este nuevo camino es mejor
                    if hijo.en_lista(nodos_frontera):
                        for n in nodos_frontera:
                            if n.igual(hijo) and n.get_coste() > hijo.get_coste():
                                nodos_frontera.remove(n)
                                nodos_frontera.append(hijo)
                        else:
                            nodos_frontera.append(hijo)

            nodo.set_hijos(lista_hijos)

    return None
```

figura 13 ejecución de ejercicio en colab

```
# Configuración del grafo y prueba del algoritmo
if __name__ == "__main__":
    # Diccionario de conexiones basado en el ejercicio de clase
    conexiones = {
        "H": {"G": 3, "B": 6, "F": 9, "A": 10, "D": 14},
        "G": {"B": 6, "C": 9, "E": 15},
        "B": {"A": 3, "C": 5, "D": 8, "E": 4},
        "A": {"D": 2, "C": 5},
        "F": {"C": 7},
        "C": {"E": 1},
        "D": {},
        "E": {}
    }

    inicio = "H"
    fin = "E"

    nodo_solucion = buscar_solucion_UCS(conexiones, inicio, fin)

    # Resultados finales
    if nodo_solucion:
        resultado = []
        n = nodo_solucion
        while n.get_padre() is not None:
            resultado.append(n.get_datos())
            n = n.get_padre()
        resultado.append(inicio)
        resultado.reverse()

        print("Ruta es : " + str(resultado))
        print("Coste: " + str(nodo_solucion.get_coste()))
    else:
        print("No se encontró una ruta.")

... Ruta es : ['H', 'B', 'E']
Coste: 10
```

figura 14 resultado ejecutado en colab

EJERCICIO 4

Encuentre la ruta desde 0 hasta 8 aplicando:

- Búsqueda en amplitud
- Búsqueda en profundidad

Establezca prioridades por orden numérico

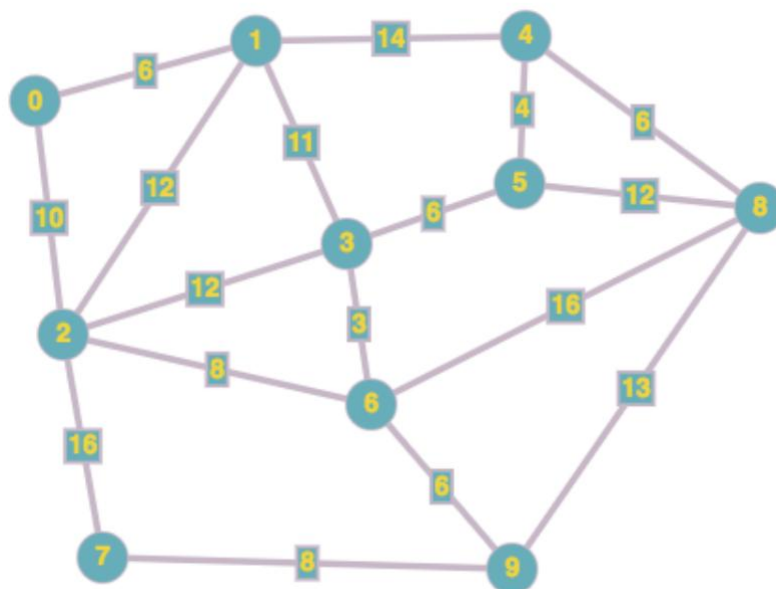


figura 15 Ejercicio 4

Estado	Expandido
{ 0 }	{ 0 }
{ 10, 20 }	{ 10 }
{ 20, 31, 41 }	{ 20 }
{ 31, 41, 62, 72 }	{ 31 }
{ 41, 62, 72, 53 }	{ 41 }
{ 62, 72, 53, 84 }	{ 62 }
{ 72, 53, 84, 96 }	{ 72 }
{ 53, 84, 96 }	{ 53 }
{ 84, 96 }	{ 84 }
Estados=9	Ruta = 0,1,4,8

figura 16 resultado búsqueda amplitud

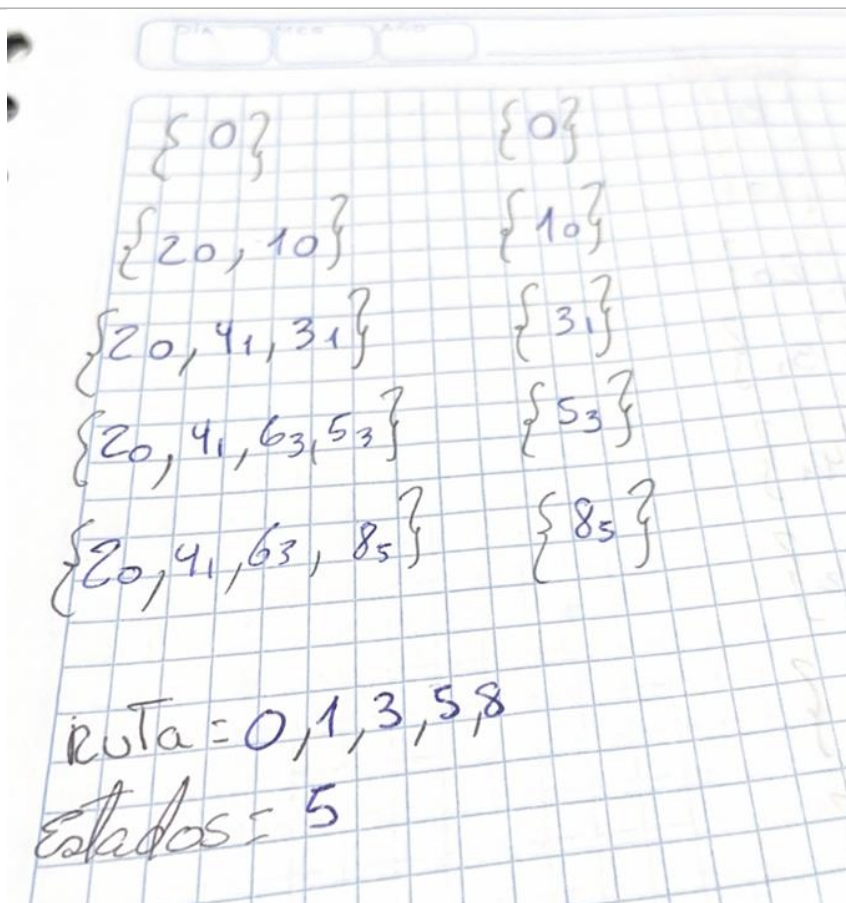


figura 17 resultado búsqueda profundidad

```
# Definición del grafo basado en la imagen (lista de adyacencia)
grafo_nodos = {
    0: [1, 2],
    1: [0, 2, 3, 4],
    2: [0, 1, 3, 6, 7],
    3: [1, 2, 5, 6],
    4: [1, 5, 8],
    5: [3, 4, 8],
    6: [2, 3, 8, 9],
    7: [2, 9],
    8: [4, 5, 6, 9],
    9: [6, 7, 8]
}

# --- Búsqueda en Amplitud (BFS) ---
def busqueda_amplitud(grafo, inicio, meta):
    # La cola usa comportamiento FIFO
    cola = [(inicio, [inicio])]
    visitados = {inicio}

    estados_expandidos = 0

    while cola:
        nodo_actual, ruta = cola.pop(0)
        estados_expandidos += 1

        if nodo_actual == meta:
            return ruta, estados_expandidos

        # Ordenamos los vecinos de menor a mayor para dar prioridad numérica
        vecinos = sorted(grafo[nodo_actual])

        for vecino in vecinos:
            if vecino not in visitados:
                visitados.add(vecino)
                cola.append((vecino, ruta + [vecino]))

    return None, estados_expandidos

# --- Búsqueda en Profundidad (DFS) ---
def busqueda_profundidad(grafo, inicio, meta):
    pila = [(inicio, [inicio])]
    visitados = {inicio}
```

```

estados_expandidos = 0

while pila:
    nodo_actual, ruta = pila.pop()
    estados_expandidos += 1

    if nodo_actual == meta:
        return ruta, estados_expandidos

    # Para que el nodo menor quede "arriba" en la pila y salga primero,
    # insertamos los vecinos ordenados de MAYOR a MENOR.
    vecinos = sorted(grafo[nodo_actual], reverse=True)

    for vecino in vecinos:
        if vecino not in visitados:
            visitados.add(vecino)
            pila.append((vecino, ruta + [vecino]))

    return None, estados_expandidos

# Ejecución de los algoritmos desde 0 hasta 8
ruta_bfs, expandidos_bfs = busqueda_amplitud(grafo_nodos, 0, 8)
print("\n1. Búsqueda en Amplitud (BFS):")
print(f"Ruta encontrada: {ruta_bfs}")
print(f"Estados expandidos: {expandidos_bfs}")

ruta_dfs, expandidos_dfs = busqueda_profundidad(grafo_nodos, 0, 8)
print("\n2. Búsqueda en Profundidad (DFS):")
print(f"Ruta encontrada: {ruta_dfs}")
print(f"Estados expandidos: {expandidos_dfs}")

```

```

1. Búsqueda en Amplitud (BFS):
Ruta encontrada: [0, 1, 4, 8]
Estados expandidos: 9

2. Búsqueda en Profundidad (DFS):
Ruta encontrada: [0, 1, 3, 5, 8]
Estados expandidos: 5

```

figura 19 resultado amplitud, profundidad

EJERCICIO 5

Aplique el algoritmo de Floyds para encontrar la ruta y la distancia desde el estado R hasta el estado d.

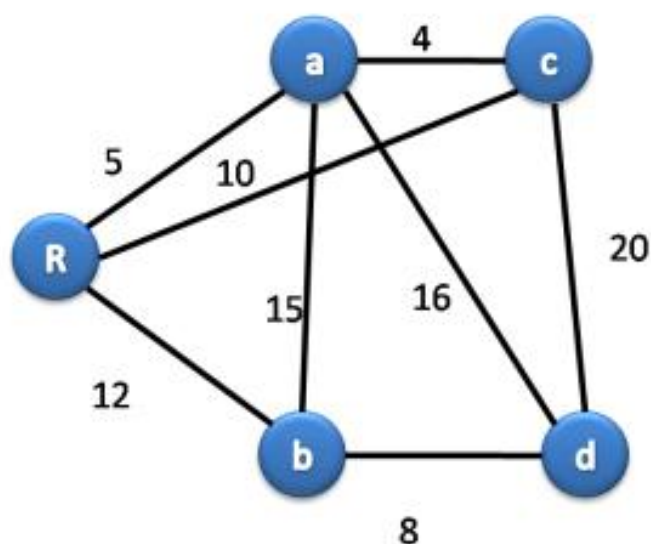


figura 20 ejercicio 5

	R	a	b	c	d	
R	0	5	12	10	∞	Ruta = R, b, d distancia = 20
a	5	0	15	4	16	
b	12	15	0	∞	8	
c	10	4	∞	0	20	
d	∞	16	8	20	0	

figura 21 algoritmo de floyds matriz

```
import math

# Asignamos math.inf a la variable 'i'
i = math.inf

vertices = ['R', 'a', 'b', 'c', 'd']

costos = [
    [0, 5, 12, 10, i],
    [5, 0, 15, 4, 16],
    [12, 15, 0, i, 8],
    [10, 4, i, 0, 20],
    [i, 16, 8, 20, 0]
]

prox_nodo = [[None] * len(vertices) for _ in range(len(vertices))]

for fila in range(len(vertices)):
    for col in range(len(vertices)):
        if costos[fila][col] != i and fila != col:
            prox_nodo[fila][col] = col

# ALGORITMO DE FLOYD-WARSHALL
for k in range(len(vertices)):
    for fila in range(len(vertices)):
        for col in range(len(vertices)):
            if costos[fila][k] + costos[k][col] < costos[fila][col]:
                costos[fila][col] = costos[fila][k] + costos[k][col]
                prox_nodo[fila][col] = prox_nodo[fila][k]

def hallar_camino(origen, destino):
    idx_origen = vertices.index(origen)
    idx_destino = vertices.index(destino)

    if prox_nodo[idx_origen][idx_destino] is None:
        return []

    camino = [origen]
    while idx_origen != idx_destino:
        idx_origen = prox_nodo[idx_origen][idx_destino]
```

```
        camino.append(vertices[idx_origen])
    return camino

# EJECUCIÓN
origen = 'R'
destino = 'd'
camino_final = hallar_camino(origen, destino)
costo_total = costos[vertices.index(origen)][vertices.index(destino)]

print("Ruta mínima:", " -> ".join(camino_final))
print("Distancia mínima:", costo_total)
```

• Ruta mínima: R -> b -> d
Distancia mínima: 20

figura 22 resultado algoritmo de floyds en colab

EJERCICIO 6

Encuentre la ruta óptima desde 1 hasta 12 aplicando los siguientes algoritmos:

Búsqueda de coste uniforme

Búsqueda A*

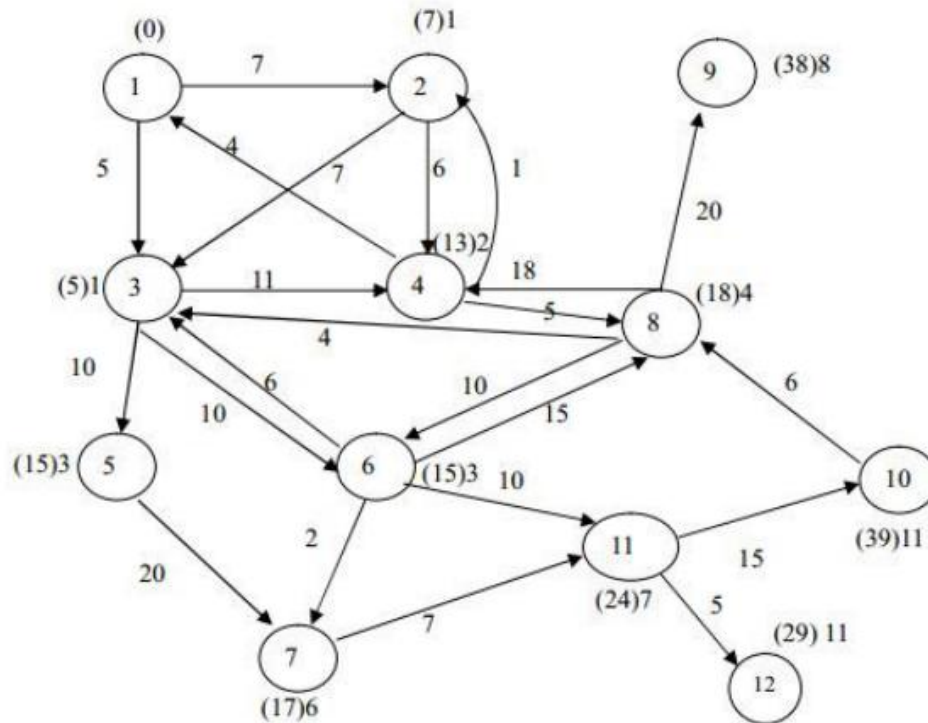


figura 23 ejercicio 6

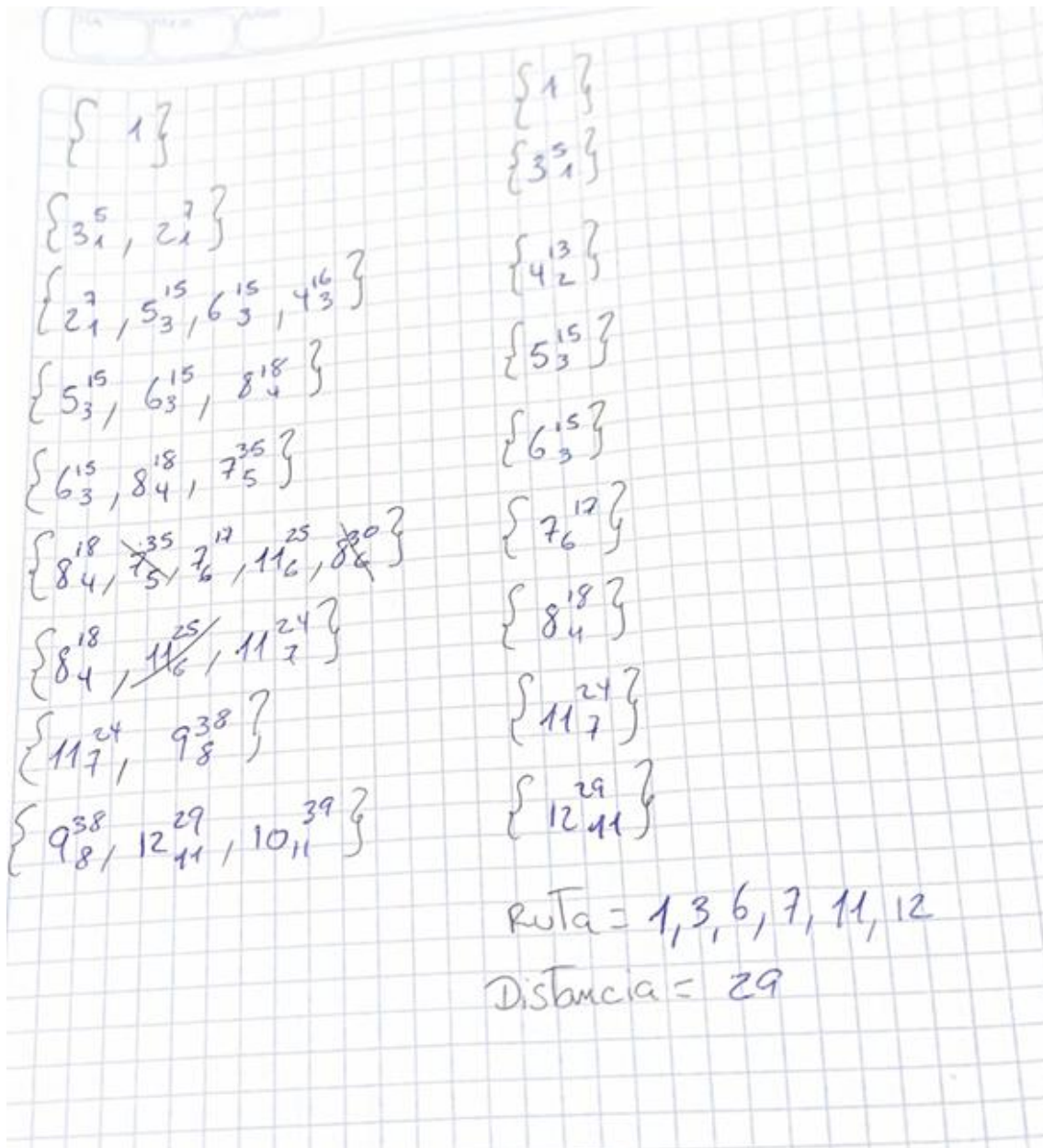


figura 24 Coste uniforme

Búsqueda A*

$$\{1\}$$

$$\{1\}$$

$$\{2^{7+7}, 3^{5+5}\}$$

$$\{3^5\}$$

$$\{4^{16+3}, 5^{15+15}, 6^{15+15}\}$$

$$\{2^7\}$$

$$\{3^{14+5}, 4^{13+13}\}$$

$$\{4^{13}\}$$

$$\{8^{18+18}\}$$

$$\{5^{15}\}$$

$$\{7^{25+17}\}$$

$$\{6^{15}\}$$

$$\{8^{30+18}, 7^{17+17}, 11^{25+24}\}$$

$$\{7^{17}\}$$

$$\{11^{24+24}\}$$

$$\{8^{18}\}$$

$$\{9^{38+38}\}$$

$$\{11^{24}\}$$

$$\{10^{39+39}, 12^{29+29}\}$$

$$\{12^{29}\}$$

Distancia : 29

Ruta : $\{1, 3, 6, 7, 11, 12\}$

figura 25 Búsqueda A*

```
import heapq

# Estructura del grafo con conexiones y pesos
red_conexiones = {
    1: {2: 7, 3: 5},
    2: {3: 7, 4: 6},
    3: {4: 11, 5: 10, 6: 10},
    4: {1: 4, 2: 1, 8: 5},
    5: {7: 20},
    6: {3: 6, 7: 2, 8: 15, 11: 10},
    7: {11: 7},
    8: {3: 4, 4: 18, 6: 10, 9: 20},
    9: {},
    10: {8: 6},
    11: {10: 15, 12: 5},
    12: {}
}

# Valores de la función heurística h(n)
valores_h = {
    1: 0,
    2: 7,
    3: 5,
    4: 13,
    5: 15,
    6: 15,
    7: 17,
    8: 18,
    9: 38,
    10: 39,
    11: 24,
    12: 0,
}

# --- Búsqueda de costo uniforme (UCS) ---
def algoritmo_ucs(grafo, nodo_inicio, nodo_meta):
    cola_pendientes = []
    heapq.heappush(cola_pendientes, (0, nodo_inicio, [nodo_inicio]))
    visitados_ucs = set()

    while cola_pendientes:
```

figura 26 Búsqueda coste uniforme

```
# --- Búsqueda de costo uniforme
def algoritmo_ucs(grafo, nodo_inicio, nodo_meta):
    cola_pendientes = []
    heapq.heappush(cola_pendientes, (0, nodo_inicio, [nodo_inicio]))
    visitados_ucs = set()

    while cola_pendientes:
        distancia_acumulada, vertice_actual, trayectoria = heapq.heappop(cola_pendientes)

        if vertice_actual == nodo_meta:
            return trayectoria, distancia_acumulada

        if vertice_actual in visitados_ucs:
            continue
        visitados_ucs.add(vertice_actual)

        for vecino, costo_arista in grafo[vertice_actual].items():
            if vecino not in visitados_ucs:
                nueva_distancia = distancia_acumulada + costo_arista
                heapq.heappush(cola_pendientes, (nueva_distancia, vecino, trayectoria + [vecino]))

    return None, float('inf')

# Ejecución de ejemplo UCS
print("--- Búsqueda de Costo Uniforme")
ruta_ucs, coste_ucs = algoritmo_ucs(red_conexiones, 1, 12)
print("Trayectoria UCS:", ruta_ucs)
print("Distancia total:", coste_ucs)
print("\n")

--- Búsqueda de Costo Uniforme
Trayectoria UCS: [1, 3, 6, 7, 11, 12]
Distancia total: 29
```

figura 27 resultado de coste uniforme

```
# --- Búsqueda A*
def algoritmo_a_star(grafo, heuristica, nodo_inicio, nodo_meta):
    # Frontera almacena: (valor_f, vertice_actual, valor_g, trayectoria)
    frontera = []
    heapq.heappush(frontera, (heuristica[nodo_inicio], nodo_inicio, 0, [nodo_inicio]))

    nodos_cerrados = set()

    while frontera:
        valor_f, vertice_actual, valor_g, trayectoria = heapq.heappop(frontera)

        if vertice_actual == nodo_meta:
            return trayectoria, valor_g

        if vertice_actual in nodos_cerrados:
            continue
        nodos_cerrados.add(vertice_actual)

        for vecino, costo_arista in grafo[vertice_actual].items():
            if vecino in nodos_cerrados:
                continue
            nuevo_valor_g = valor_g + costo_arista
            nuevo_valor_f = nuevo_valor_g + heuristica.get(vecino, float('inf'))
            heapq.heappush(frontera, (nuevo_valor_f, vecino, nuevo_valor_g, trayectoria + [vecino]))

    return None, float('inf')

# Ejecución de ejemplo A*
print(" Búsqueda A*")
ruta_astar, coste_astar = algoritmo_a_star(red_conexiones, valores_h, 1, 12)
print("Trayectoria A*:", ruta_astar)
print("Distancia total:", coste_astar)

Búsqueda A*
Trayectoria A*: [1, 3, 6, 7, 11, 12]
Distancia total: 29
```

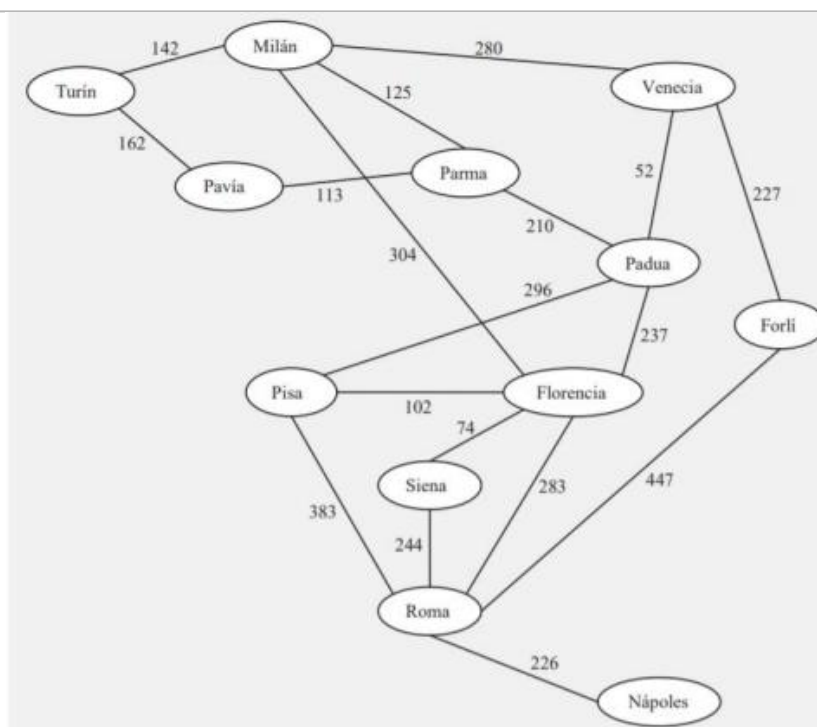
*figura 28 resultado A**

Ejercicio

7

Mediante A* halle la ruta más corta de Pavia a Nápoles

Mediante Coste Uniforme halle la Ruta más corta de Turín a Roma



<i>Turín</i>	<i>893</i>	<i>Pisa</i>	<i>571</i>
<i>Milán</i>	<i>774</i>	<i>Forlì</i>	<i>557</i>
<i>Pavía</i>	<i>764</i>	<i>Florencia</i>	<i>473</i>
<i>Venecia</i>	<i>734</i>	<i>Siena</i>	<i>437</i>
<i>Parma</i>	<i>658</i>	<i>Roma</i>	<i>226</i>
<i>Padua</i>	<i>695</i>	<i>Nápoles</i>	<i>0</i>

figura 29 ejercicio 7

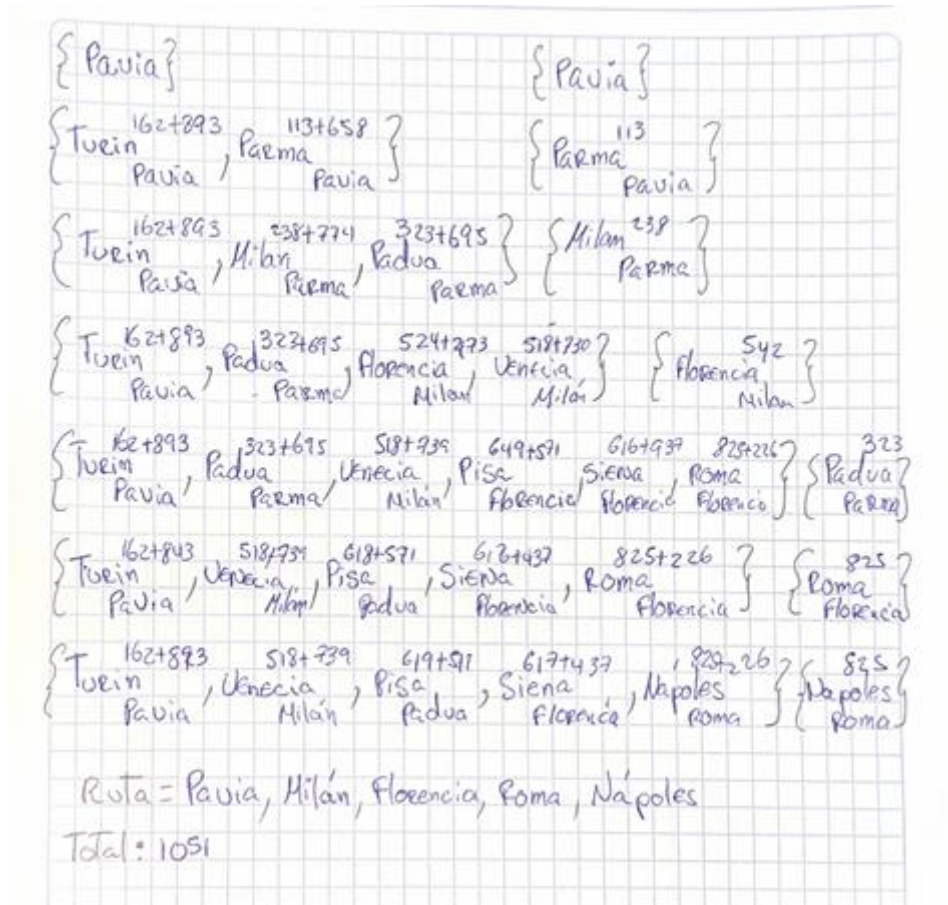


figura 30 ejercicio mediante A*



figura 31 mediante coste uniforme

```
def algoritmo_a_star(grafo, inicio, meta, heuristica):
    nodos_pendientes = [(heuristica[inicio], inicio, 0, [inicio])]
    nodos_cerrados = set()

    while nodos_pendientes:
        costo_f, nodo_actual, costo_g_actual, trayectoria = min(nodos_pendientes)
        nodos_pendientes.remove((costo_f, nodo_actual, costo_g_actual, trayectoria))

        if nodo_actual == meta:
            return trayectoria, costo_g_actual

        if nodo_actual in nodos_cerrados:
            continue

        nodos_cerrados.add(nodo_actual)

        for nodo_adyacente, peso_arista in grafo[nodo_actual].items():
            if nodo_adyacente not in nodos_cerrados:
                nuevo_costo_g = costo_g_actual + peso_arista
                nuevo_costo_f = nuevo_costo_g + heuristica[nodo_adyacente]
                nueva_trayectoria = trayectoria + [nodo_adyacente]
                nodos_pendientes.append((nuevo_costo_f, nodo_adyacente, nuevo_costo_g, nueva_trayectoria))

    return None, float('inf')
```

```
# Grafo con distancias entre ciudades
red_vial_italia = {
    "Turín": {"Milán": 142, "Pavia": 162},
    "Milán": {"Turín": 142, "Venecia": 280, "Parma": 125, "Florencia": 304},
    "Venecia": {"Milán": 280, "Padua": 52, "Forlì": 227},
    "Parma": {"Milán": 125, "Padua": 210, "Pavia": 103},
    "Pavia": {"Turín": 162, "Parma": 113},
    "Padua": {"Venecia": 52, "Parma": 210, "Pisa": 296, "Florencia": 283},
    "Forlì": {"Venecia": 227, "Roma": 447},
    "Pisa": {"Padua": 296, "Florencia": 102, "Roma": 383},
    "Florencia": {"Padua": 237, "Pisa": 102, "Siena": 74, "Roma": 283},
    "Siena": {"Florencia": 74, "Roma": 244},
    "Roma": {"Forlì": 447, "Pisa": 383, "Florencia": 283, "Siena": 244, "Nápoles": 226},
    "Nápoles": {"Roma": 226}
}

# Estimación de distancia restante hasta Nápoles
distancia_linea_recta_napoles = {
    "Turín": 893,
    "Milán": 774,
    "Pavia": 764,
    "Venecia": 734,
    "Parma": 658,
    "Padua": 695,
    "Pisa": 571,
    "Forlì": 557,
    "Florencia": 473,
    "Siena": 437,
    "Roma": 226,
    "Nápoles": 0
}

# Ejecutar búsqueda desde Pavia hasta Nápoles
ruta_final, costo_final = algoritmo_a_star(red_vial_italia, "Pavia", "Nápoles", distancia_linea_recta_napoles)

print("Ruta más corta de Pavia a Nápoles Mediante A*:")
print(" -> ".join(ruta_final))
print(f"Costo total Mediante Coste Uniforme: {costo_final} kilómetros")

Ruta más corta de Pavia a Nápoles Mediante A*:
Pavia -> Parma -> Milán -> Florencia -> Roma -> Nápoles
Costo total Mediante Coste Uniforme: 1051 kilómetros
```

figura 32 resultado coste uniforme

EJERCICIO 8

Aplique el algoritmo de Dijkstra para hallar la ruta más corta del estado 1 hasta 20

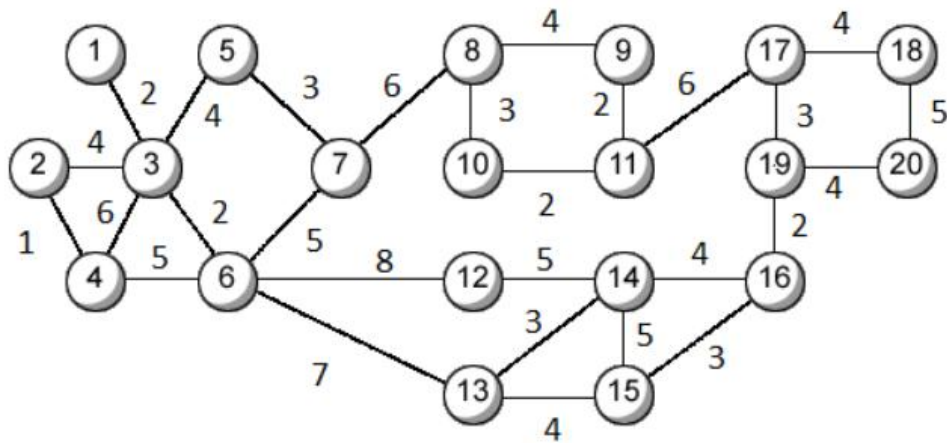


figura 33 ejercicio 8

```
import heapq

def encontrar_camino_mas_corto(red_nodos, origen):
    costos_acumulados = {nodo: float('inf') for nodo in red_nodos}
    nodos_anteriores = {nodo: None for nodo in red_nodos}
    costos_acumulados[origen] = 0
    cola_prioridad = [(0, origen)]

    while cola_prioridad:
        costo_actual, nodo_actual = heapq.heappop(cola_prioridad)

        if costo_actual > costos_acumulados[nodo_actual]:
            continue

        for vecino, costo_tramo in red_nodos[nodo_actual]:
            nuevo_costo = costo_actual + costo_tramo
            if nuevo_costo < costos_acumulados[vecino]:
                costos_acumulados[vecino] = nuevo_costo
                nodos_anteriores[vecino] = nodo_actual
                heapq.heappush(cola_prioridad, (nuevo_costo, vecino))

    return costos_acumulados, nodos_anteriores

# Reconstrucción del camino más corto
def trazar_ruta(nodos_anteriores, destino):
    trayecto_final = []
    while destino:
        trayecto_final.append(destino)
        destino = nodos_anteriores[destino]
    return trayecto_final[::-1]
```

figura 34 ejercicio 8

```

mapa_rutas = {
    1: [(3, 2)],
    2: [(3, 4), (4, 1)],
    3: [(1, 2), (2, 4), (4, 6), (5, 4), (6, 2)],
    4: [(2, 1), (3, 6), (6, 5)],
    5: [(3, 4), (7, 3)],
    6: [(3, 2), (4, 5), (7, 5), (12, 50), (13, 50)],
    7: [(5, 3), (6, 5), (8, 6)],
    8: [(7, 6), (9, 4), (10, 3)],
    9: [(8, 4), (11, 2)],
    10: [(8, 3), (11, 2)],
    11: [(9, 2), (10, 2), (17, 6)],
    12: [(6, 50), (14, 5)],
    13: [(6, 50), (14, 3), (15, 4)],
    14: [(12, 5), (13, 3), (15, 5), (16, 4)],
    15: [(13, 4), (14, 5), (16, 3)],
    16: [(14, 4), (15, 3), (19, 2)],
    17: [(11, 6), (18, 4), (19, 3)],
    18: [(17, 4), (20, 5)],
    19: [(16, 2), (17, 3), (20, 4)],
    20: [(18, 5), (19, 4)]
}

distancias_finales, rastreo_nodos = encontrar_camino_mas_corto(mapa_rutas, 1)
ruta_optima = trazar_ruta(rastreo_nodos, 20)

# Mostrar resultados
print(f"Distancia mínima del nodo 1 al 20: {distancias_finales[20]}")
print(f"Ruta más corta: {ruta_optima}")

• Distancia mínima del nodo 1 al 20: 33
  Ruta más corta: [1, 3, 6, 7, 8, 10, 11, 17, 19, 20]

```

figura 35 resultado de algoritmo de Dijkstra

EJERCICIO 9

En base a la siguiente tabla, aplique el método de sobrante estocástico para determinar el número de veces que se reproduce cada cromosoma.

i	f(xi)	vi	ei	di	ri	d>r
1	12.14					
2	37.85					
3	19.75					
4	33.42					
5	25.64					
6	35.15					
7	38.94					

8	32.07					
9	27.51					
10	25.61					

```
import random

valores Aptitud = [11.96, 32.20, 17.25, 35.51, 27.49, 38.25, 34.63, 39.93, 26.43, 23.71]
tamano poblacion = 10
suma Aptitudes = round(sum(valores Aptitud), 2)

valores esperados = [valor / (suma Aptitudes / tamano poblacion) for valor in valores Aptitud]
parte Entera = [int(esperado // 1) for esperado in valores esperados]
parte Fraccionaria = [esperado % 1 for esperado in valores esperados]

numeros Azar = [0.4316, 0.0287, 0.7168, 0.5171, 0.8188, 0.8581, 0.0876, 0.5616, 0.5192, 0.5390]
copias Extra = [1 if parte Fraccionaria[i] > numeros Azar[i] else 0 for i in range(tamano poblacion)]

print(f'Suma de Aptitudes: {suma Aptitudes}')
print(f'Valores Esperados: {valores esperados}')
print(f'Parte Entera: {parte Entera}')
print(f'Parte Fraccionaria: {parte Fraccionaria}')
print(f'Números al Azar: {numeros Azar}')
print(f'Copias Extra: {copias Extra}')
```

figura 36 resultado de método sobrante estocástico

```
Suma de Aptitudes: 287.36
Valores Esperados: [0.4162026726057907, 1.1205456570155903, 0.6002923162583519, 1.2357321826280623, 0.9566397550111357, 1.3310829621380846, 1.205108574610245, 1.389546213808463, 0.9197522271714922, 0.825097438752784]
Parte Entera: [0, 1, 0, 1, 0, 1, 1, 1, 0, 0]
Parte Fraccionaria: [0.4162026726057907, 0.12054565701559028, 0.6002923162583519, 0.23573218262806228, 0.9566397550111357, 0.33108296213808464, 0.205108574610245, 0.3895462138084631, 0.9197522271714922, 0.825097438752784]
Números al Azar: [0.4316, 0.0287, 0.7168, 0.5171, 0.8188, 0.8581, 0.0876, 0.5616, 0.5192, 0.539]
Copias Extra: [0, 1, 0, 0, 1, 0, 1, 0, 1, 1]
```

figura 37 resultado de método sobrante estocástico

EJERCICIO 10

Realice la primera iteración con todas las etapas (Selección, Cruce y Mutación), para la selección utilice el método de la Ruleta sesgada. El cruce es de dos puntos con $U1=3$ y $U2=6$, la mutación es de tipo I y se da en los bits 2 y 5 del cromosoma 3 y 6

Considere predisposición

$$F(x) = Ae^{\frac{(x-b)^2}{c^2}}$$

Donde: $A=14$, $b=84$, $c=32$

Considere la población inicial:

$$P_0 = \begin{pmatrix} 10101101 \\ 10011010 \\ 01100001 \\ 11011001 \\ 01011110 \end{pmatrix}$$

```
import numpy as np
import pandas as pd

# 1. Parámetros de la función de predisposición
A = 14
b = 84
c = 32

# Función de predisposición
def calcular_apititud(x):
    return A * np.exp(-(x - b) ** 2) / (c ** 2)

# 2. Población inicial P0 (binaria)
poblacion_inicial = [
    '10101101',
    '10011010',
    '01100001',
    '11011001',
    '01011110'
]

# Decodificación y evaluación
valores_decimales = [int(cromosoma, 2) for cromosoma in poblacion_inicial]
valores_apititud = [calcular_apititud(x) for x in valores_decimales]
suma_total_apititud = sum(valores_apititud)

# Probabilidades para la Ruleta Sesgada
probabilidades_ruleta = [apt / suma_total_apititud for apt in valores_apititud]

np.random.seed(0) # Semilla para reproducibilidad
indices_seleccionados = np.random.choice(len(poblacion_inicial), size=len(poblacion_inicial), p=probabilidades_ruleta)
poblacion_post_seleccion = [poblacion_inicial[i] for i in indices_seleccionados]

# Cruce de dos puntos (U1=3, U2=6)
punto_cruce_1, punto_cruce_2 = 3, 6
poblacion_post_cruce = []
```

figura 38 ejecución de ruleta sesgada

```
for i in range(0, len(poblacion_post_seleccion) - 1, 2):
    padre_1 = poblacion_post_seleccion[i]
    padre_2 = poblacion_post_seleccion[i + 1]

    # Intercambio de segmentos genéticos
    hijo_1 = padre_1[:punto_cruce_1] + padre_2[punto_cruce_1:punto_cruce_2] + padre_1[punto_cruce_2:]
    hijo_2 = padre_2[:punto_cruce_1] + padre_1[punto_cruce_1:punto_cruce_2] + padre_2[punto_cruce_2:]

    poblacion_post_cruce.extend([hijo_1, hijo_2])

# Si la población es impar, el último individuo pasa directamente
if len(poblacion_post_seleccion) % 2 != 0:
    poblacion_post_cruce.append(poblacion_post_seleccion[-1])

# Mutación tipo I Invertir bits 2 y 5 -> Índices 1 y 4 por ser base 0
indices_bits_mutar = [1, 4]

def aplicar_mutacion_tipo_1(cromosoma, posiciones):
    cromosoma_lista = list(cromosoma)
    for pos in posiciones:
        cromosoma_lista[pos] = '0' if cromosoma_lista[pos] == '1' else '1'
    return ''.join(cromosoma_lista)

poblacion_post_mutacion = poblacion_post_cruce.copy()

# Mutar cromosoma 3
if len(poblacion_post_mutacion) >= 3:
    poblacion_post_mutacion[2] = aplicar_mutacion_tipo_1(poblacion_post_mutacion[2], indices_bits_mutar)

# Mutar cromosoma 6
if len(poblacion_post_mutacion) >= 6:
    poblacion_post_mutacion[5] = aplicar_mutacion_tipo_1(poblacion_post_mutacion[5], indices_bits_mutar)

# RESULTADOS
df_evaluacion_inicial = pd.DataFrame({
    "Cromosoma Binario": poblacion_inicial,
    "Valor Decimal (x)": valores_decimales,
    "Aptitud f(x)": valores_aptitud,
    "Probabilidad Ruleta": probabilidades_ruleta,
```

figura 39 ejecución de ruleta sesgada

```
df_operadores_geneticos = pd.DataFrame({
    "Post-Selección": poblacion_post_seleccion,
    "Post-Cruce": poblacion_post_cruce,
    "Post-Mutación": poblacion_post_mutacion
})

print("--- ETAPA 1: EVALUACIÓN Y PROBABILIDADES ---")
print(df_evaluacion_inicial)
print("\n--- ETAPA 2: OPERADORES GENÉTICOS (Selección, Cruce, Mutación) ---")
print(df_operadores_geneticos)
```

```

--- ETAPA 1: EVALUACIÓN Y PROBABILIDADES ---
  Cromosoma Binario  Valor Decimal (x)  Aptitud f(x)  Probabilidad Ruleta
0          10101101          173  6.119382e-03      2.478430e-04
1          10011010          154  1.169395e-01      4.736202e-03
2          01100001           97  1.187005e+01      4.807526e-01
3          11011001          217  4.404994e-07      1.784081e-08
4          01011110           94  1.269745e+01      5.142634e-01

--- ETAPA 2: OPERADORES GENÉTICOS (Selección, Cruce, Mutación) ---
  Post-Selección Post-Cruce Post-Mutación
0          01011110  01011110  01011110
1          01011110  01011110  01011110
2          01011110  01011110  00010110
3          01011110  01011110  01011110
4          01100001  01100001  01100001

```

figura 40 resultado de ejercicio 10

EJERCICIO 11

Realice la primera iteración con todas las etapas (Selección, Cruce y Mutación), para la selección utilice el método de la Ruleta Bernoulli. El cruce es uniforme con la máscara $M=10110110$, la mutación es de tipo II y se da en los bits 3 y 6 del cromosoma 2 y 4.

Considere

$$f(x) = x^2 + 3x$$

$$I = [-3, 3]$$

$$v_{float} = \frac{\max_{float} - \min_{float}}{\max_{dec} - \min_{dec}} (v_{dec} - \min_{dec}) + \min_{float}$$

Considere la población inicial:

$$P_0 = \begin{pmatrix} 11010001 \\ 01001010 \\ 01101111 \\ 11101100 \\ 10101001 \end{pmatrix}$$

```
import random
from collections import Counter

def normalizar_valor(x):
    return 6 / 255 * (x - 3)

def funcion_objetivo(x):
    return x ** 2 + 3 * x

def evaluar_poblacion(poblacion):
    valores_decimales = [int("".join(map(str, individuo)), 2) for individuo in poblacion]
    print(valores_decimales)

    valores_normalizados = [normalizar_valor(x) for x in valores_decimales]
    print(valores_normalizados)

    valores_aptitud = [funcion_objetivo(x) for x in valores_normalizados]
    print(valores_aptitud)

    aptitud_total = sum(valores_aptitud)
    print(aptitud_total)

    probabilidades_seleccion = [x / aptitud_total for x in valores_aptitud]
    return probabilidades_seleccion, valores_aptitud

def seleccion_ruleta(poblacion, probabilidades_seleccion):
    conteo_reproducciones = {probabilidad: 0 for probabilidad in probabilidades_seleccion}
    individuos_seleccionados = 1

    while individuos_seleccionados <= len(conteo_reproducciones):
        for probabilidad in conteo_reproducciones:
            numero_azar = random.random()
            print(f"{probabilidad} >= {numero_azar}")
```

Figura 41 ejecución de ruleta Bernoulli.

```

        if numero_azar <= probabilidad:
            print("¡Seleccionado!")
            conteo_reproducciones[probabilidad] += 1
            individuos_seleccionados += 1

        if individuos_seleccionados > len(conteo_reproducciones):
            break

    poblacion_seleccionada = []
    print(conteo_reproducciones)

    for i, (probabilidad, repeticiones) in enumerate(conteo_reproducciones.items()):
        for _ in range(repeticiones):
            poblacion_seleccionada.append(poblacion[i])

    return poblacion_seleccionada

def cruce_uniforme_con_mascara(padre_1, padre_2, mascara):
    hijo_1 = []
    hijo_2 = []

    for i in range(len(mascara)):
        if mascara[i] == 1:
            hijo_1.append(padre_1[i])
            hijo_2.append(padre_2[i])
        else:
            hijo_1.append(padre_2[i])
            hijo_2.append(padre_1[i])

    return hijo_1, hijo_2

def generar_descendencia(poblacion_seleccionada, mascara):
    frecuencias_cromosomas = Counter(tuple(cromosoma) for cromosoma in poblacion_seleccionada)
    cromosoma_mas_comun, _ = frecuencias_cromosomas.most_common(1)[0]
    print(f"Cromosoma más común: {cromosoma_mas_comun}")

```

Figura 42 ejecución de ruleta de Bernoulli.

```

    for cromosoma_actual in cromosomas_a_mutar:
        print(f"Cromosoma original: {cromosoma_actual}")
        cromosoma_mutado = aplicar_mutacion_tipo_2(cromosoma_actual, posiciones_a_mutar, probabilidad_mutacion)
        print(f"Cromosoma mutado: {cromosoma_mutado}\n")

# Datos de prueba
poblacion_inicial = [
    [1, 1, 0, 1, 0, 0, 0, 1],
    [0, 1, 1, 0, 1, 0, 1, 0],
    [0, 1, 0, 0, 1, 1, 1, 1],
    [1, 0, 1, 1, 0, 1, 1, 0],
    [1, 0, 1, 0, 1, 1, 0, 1]
]

```

Figura 43 ejecución de ruleta de Bernoulli

```

padre_principal = list(cromosoma_mas_comun)
hijos_generados = []

for individuo in poblacion_seleccionada:
    if tuple(individuo) != cromosoma_mas_comun:
        hijo_1, hijo_2 = cruce_uniforme_con_mascara(padre_principal, individuo, mascara)
        print(f"Cruce entre {padre_principal} y {individuo} con máscara {mascara} -> Hijo 1: {hijo_1}, Hijo 2: {hijo_2}")
        hijos_generados.extend([hijo_1, hijo_2])

return hijos_generados

import random

def aplicar_mutacion_tipo_2(individuo, posiciones_a_mutar, probabilidad_mutacion):
    cromosoma_mutado = individuo.copy()

    for i in posiciones_a_mutar:
        numero_azar = random.random()
        # Se formatea a 4 decimales para mayor legibilidad en consola
        print(f"numero_azar:.4f" <= {probabilidad_mutacion:.4f} = {numero_azar <= probabilidad_mutacion}")

        if numero_azar <= probabilidad_mutacion:
            print(f"Posición {i+1} del cromosoma mutada a 0")
            cromosoma_mutado[i] = 0
        else:
            print(f"Posición {i+1} del cromosoma mutada a 1")
            cromosoma_mutado[i] = 1

    return cromosoma_mutado

def procesar_fase_mutacion(descendencia, posiciones_a_mutar):
    # Se genera una probabilidad global para toda la fase de mutación
    probabilidad_mutacion = random.random()
    print(f"Probabilidad de mutación generada: {probabilidad_mutacion:.4f}\n")

    # Se seleccionan el tercer y quinto individuo de la lista
    cromosomas_a_mutar = [descendencia[2], descendencia[4]]

```

figura 44 ejecución de ruleta de Bernoulli

```

mascara_cruce = [1, 0, 1, 1, 0, 1, 1, 0]
descendencia = generar_descendencia(poblacion_seleccionada, mascara_cruce)

print(descendencia)

Cromosoma más común: (1, 1, 0, 1, 0, 0, 0, 1)
Cruce entre [1, 1, 0, 1, 0, 0, 0, 1] y [0, 1, 1, 0, 1, 0, 1, 0] con máscara [1, 0, 1, 1, 0, 1, 1, 0] -> Hijo 1: [1, 1, 0, 1, 1, 0, 0, 0], Hijo 2: [0, 1, 1, 0, 0, 0, 1, 1]
Cruce entre [1, 1, 0, 1, 0, 0, 0, 1] y [1, 0, 1, 1, 0, 1, 1, 0] con máscara [1, 0, 1, 1, 0, 1, 1, 0] -> Hijo 1: [1, 0, 0, 1, 0, 0, 0, 0], Hijo 2: [1, 1, 1, 1, 0, 1, 1, 1]
Cruce entre [1, 1, 0, 1, 0, 0, 0, 1] y [1, 0, 1, 0, 1, 1, 0, 1] con máscara [1, 0, 1, 1, 0, 1, 1, 0] -> Hijo 1: [1, 0, 0, 1, 1, 0, 0, 1], Hijo 2: [1, 1, 1, 0, 0, 1, 0, 1]
[[1, 1, 0, 1, 1, 0, 0, 0], [0, 1, 1, 0, 0, 0, 1, 1], [1, 0, 0, 1, 0, 0, 0, 0], [1, 1, 1, 1, 0, 1, 1, 1], [1, 0, 0, 1, 1, 0, 0, 1], [1, 1, 1, 0, 1, 0, 1, 1]]

posiciones_a_mutar = [2, 5]
procesar_fase_mutacion(descendencia, posiciones_a_mutar)

Probabilidad de mutación generada: 0.2291

Cromosoma original: [1, 0, 0, 1, 0, 0, 0, 0]
0.5695 <= 0.2291 = False
Posición 3 del cromosoma mutada a 1
0.4162 <= 0.2291 = False
Posición 6 del cromosoma mutada a 1
Cromosoma mutado: [1, 0, 1, 1, 0, 1, 0, 0]

Cromosoma original: [1, 0, 0, 1, 1, 0, 0, 1]
0.0537 <= 0.2291 = True
Posición 3 del cromosoma mutada a 0
0.8274 <= 0.2291 = False
Posición 6 del cromosoma mutada a 1
Cromosoma mutado: [1, 0, 0, 1, 1, 1, 0, 1]

```

figura 45 resultado del ejercicio 11

Link de Colab

<https://colab.research.google.com/drive/1HfAtOmjkKTC4DcqLrSC--xPFEjscPUaC?usp=sharing>